Documentation de réponse au cahier des charges

Sommaire

Éléments implémentés et non implémentés 2
Implémentés: 2
Non Implémentés: 3
Structure du code, modules, protocole, API 3
Structure générale: 4
Description de l'algorithme de chiffrage (forces et faiblesses) 6
Principe du chiffrement César: 6
Forces 6
Faiblesses 7
Rapport de projet avec gestion du projet 7

Éléments implémentés et non implémentés

Implémentés :

Chiffrement et déchiffrement :

- **Chiffrement César** : Le chiffrement est basé sur l'algorithme César qui décale chaque caractère d'un nombre défini par une clé.

- **Algorithme de chiffrement/déchiffrement pour les messages** : Implémenté dans les fonctions chiffrer_cesar et dechiffrer_cesar.

```python
def chiffrer_cesar(texte, cle):
    resultat = ""
    for char in texte:
        if char == '|':
            resultat += '|'
        elif 'A' <= char <= 'Z':
            code = ord(char) - ord('A')
            code_chiffre = (code + cle) % 26
            resultat += chr(code_chiffre + ord('A'))
        elif 'a' <= char <= 'z':
            code = ord(char) - ord('a')
            code_chiffre = (code + cle) % 26
            resultat += chr(code_chiffre + ord('a'))
        else:
            resultat += char
    return resultat
```

```python
def dechiffrer_cesar(texte, cle):
    resultat = ""
    for char in texte:
        if char == '|':
            resultat += '|'
        elif 'A' <= char <= 'Z':
            code = ord(char) - ord('A')
            code_clair = (code - cle) % 26
            resultat += chr(code_clair + ord('A'))
        elif 'a' <= char <= 'z':
            code = ord(char) - ord('a')
            code_clair = (code - cle) % 26
            resultat += chr(code_clair + ord('a'))
        else:
            resultat += char
    return resultat
```

Routage des messages **:**

- **Encodage des messages** : La couche de routage des messages est implémentée avec les fonctions encoder_couche_routage et decoder_couche_routage. Ces fonctions s'occupent de la construction et de la déconstruction des paquets de données en format PROCHAIN|DEST|DONNEES

- **Gestion des communications entre routeurs** : Chaque routeur, à l'aide de son identifiant et de son port, enregistre des règles de routage pour envoyer et recevoir des données. Les fonctions comme transmettre_au_routeur et livrer_message_client gèrent le transfert des messages.

Base de données :

- Les informations des routeurs et des clients sont enregistrées dans une base de données MySQL à l'aide de la classe BaseDonneesRouteur. Cela inclut l'enregistrement des nœuds et des règles de routage.

Serveur Master :

- Le serveur maître gère l'enregistrement des routeurs et des clients, ainsi que la gestion des règles de routage. Il permet aussi la récupération des informations relatives aux nœuds (clients ou routeurs) via des requêtes appropriées.

Non Implémentés :

- Authentification forte des acteurs (pas de signature, pas de certificat).
- Vérification d'intégrité (pas de MAC/HMAC).
- Gestion avancée des routes en graphes : les chemins sont aléatoires et non optimisés.
- Résilience : pas de gestion de panne routeur pas de réémission automatique
- Sécurisation MySQL (pas de hash, pas de permissions limitées spécifiques)

Structure du code, modules, protocole, API

Structure générale :

1. Module Client (`client.py`)

- Interaction utilisateur
- Construction de l'oignon
- Chiffrement final
- Envoi/réception via sockets

2. Module Routeur (`Routeur.py`)

- Déchiffrement des couches
- Routage
- Livraison finale au client

3. Module Serveur Maître (`Master.py`)

- Enregistrer les noeuds
- Distribuer les informations
- Créer automatiquement les règles de routage

4. Base de données (`base_donnees.py`)

Tables :

- `noeuds(id_noeud, type_noeud, ip, port, cle_publique_e, cle_publique_n, status)`
- `table_routage(id_routeur, destination, prochain_saut, ip_suivante, port_suivant)`

API :

- `enregistrer_noeud()`
- `obtenir_info_noeud()`
- `obtenir_tous_routeurs()`
- `ajouter_regle_routage()`

5. Chiffrement (`chiffrement.py`)

Rôle principal :
Fournir toutes les opérations cryptographiques :

- Chiffrement/déchiffrement du message utilisateur
- Chiffrement des couches d'oignon entre routeurs
- Génération des clés des routeurs

Fonctionnalités essentielles :

Chiffrement César

- `chiffrer_cesar(texte, cle)`
- `dechiffrer_cesar(texte, cle)`

Utilisé :

- Par le **client** pour chiffrer le message final destiné au destinataire
- Par chaque **routeur** pour chiffrer/déchiffrer sa couche d'oignon Génération de clés
- `generer_cles()`
  Génère une clé César aléatoire entre 1 et 25 pour chaque routeur.

Chiffrement général

- `chiffrer(msg, cle_publique)`
- `dechiffrer(msg, cle_privee)`

Permet de chiffrer tout type de données avant envoi au prochain routeur.

Gestion des couches de routage

- `encoder_couche_routage(prochain, destination, donnees)`
- `decoder_couche_routage(chaine)`
- Organise la construction de l'oignon en respectant le format : `PROCHAIN_SAUT | DESTINATION | DONNEES`

Description de l'algorithme de chiffrage (forces et faiblesses)

Principe du chiffrement César:

Pour chaque caractère alphabétique :

- Convertir en position (0–25)
- Appliquer :
  `nouveau = (ancien + cle) % 26`

Pour le déchiffrement :

- `(ancien - cle) % 26`

Les caractères spéciaux sont conservés (utile pour `|` dans les couches).

Forces :

- **Très simple**
- **Coût computationnel faible**
- **Adapté pour notre projet**
- **Facile à déboguer et visualiser**

Faiblesses :

- **Sécurité extrêmement faible**
- Vulnérable :
    - Au brute-force (25 clés possibles)
    - À l'analyse fréquentielle
- Aucune confidentialité réelle dans un contexte moderne
- Pas de protection contre la modification du message
- Pas de gestion de clés sécurisée

Rapport de projet avec gestion du projet

La gestion de projet que nous avions réalisée en amont décrivait l'organisation de l'équipe, la répartition des rôles, les fonctionnalités à développer ainsi qu'un planning prévisionnel. Après finalisation du code et des tests, nous pouvons confirmer que le travail réalisé correspond globalement à ce qui avait été prévu.

Correspondance avec les fonctionnalités prévues

Les fonctionnalités essentielles identifiées dans la gestion du projet ont été effectivement implémentées :

- **Routage en oignon multi-sauts** : présent et fonctionnel,
- **Base de données MariaDB** : utilisée pour stocker nœuds et routes,
- **Gestion multithread** : opérationnelle dans les routeurs,
- **Communication client–routeur–maître** : conforme à la planification.

La seule différence notable par rapport au prévisionnel est l'abandon du chiffrement asymétrique prévu dans le fichier de gestion du projet : pour des raisons de temps et de stabilité, nous avons finalement utilisé un chiffrement César simplifié. Cette modification ne remet pas en cause l'architecture globale.